Subagents, Skills, MCPs, and Plugins in Codex and Claude Code: Creation Structures and Comparison

1. Conceptual Overview

Both Codex (OpenAI) and Claude Code (Anthropic) structure extensibility in four main layers: persistent project instructions, skills (reusable playbooks), MCP (Model Context Protocol) for external tools, and subagents/teams for delegation and parallelism. In both systems, the recommended flow is: first define global project instructions, then create skills for recurring workflows, then connect MCPs to access external systems, and only then introduce subagents or teams to parallelize specialized work.

2. Codex Customization Stack (Structure, Files, Discovery)

2.1 Logical Layers in Codex

The official guide organizes Codex into four customization layers:

- AGENTS.md: Persistent instructions that "travel" with the repository and are loaded per directory (root → current folder), defining rules, commands, conventions, and routing hints.
- **Skills**: Reusable workflows packaged in `my-skill/` directories with a mandatory `SKILL.md` and optional `scripts/`, `references/`, `assets/` folders, plus `agents/openai.yaml` for metadata and MCP dependencies.
- **MCP**: External servers (HTTP or STDIO) defined in `config.toml` under `mcp_servers.<name>`, providing access to docs, browsers, issue trackers, and more.
- **Subagents**: Specialized child agents defined by individual TOML files in `~/.codex/agents/` or `.codex/agents/`, used for parallel delegation of exploration, review, testing, triage, and bounded implementation.

These layers are complementary: AGENTS.md shapes general behavior, skills package processes, MCP connects to the "outside world," and subagents execute portions of the work in independent threads.

2.2 Codex File Paths and Configuration

Item	Default Location	Purpose
User config	`~/.codex/config.toml`	Global CLI/IDE defaults (model, sandbox, web_search, [agents], MCP, etc.)

Item	Default Location	Purpose
Project config	`.codex/config.toml`	Repository-specific overrides (only for "trusted" projects)
Custom subagents	`~/.codex/agents/` or `.codex/agents/`	One TOML file per custom agent
Global instructions	`~/.codex/AGENTS.md`	Global working style and agreements
Repo instructions	`AGENTS.md` / `AGENTS.override.md`	Per-directory rules, scanned from root → cwd
Repo skills	`.agents/skills/`	Skills visible to that repository
User skills	`$HOME/.agents/skills`	Personal skills, reusable across projects
Admin skills	`/etc/codex/skills`	Machine- or container-shared skills

Project discovery works by walking up from the current directory until a root is found (by default a directory containing `.git`, `.hg`, or `.sl`), with the ability to customize `project_root_markers` or force the cwd as root (empty list).

3. Subagents in Codex: Operational Model and Creation

3.1 Operational Model

Codex does not create subagents automatically — the user must explicitly request delegation or parallel work. When multiple subagents are created, Codex handles spawning, follow-up routing, waiting, and thread closure, returning a consolidated result at the end.

Built-in agent types exist (e.g., `default`, `worker`, `explorer`), and subagent workflows consume more tokens than single-agent runs because each worker uses its own model and tools. The `/agent` command in the CLI allows inspecting and switching between agent threads, and subagents inherit the parent's sandbox policy unless explicitly overridden.

3.2 Standalone Subagent TOML Schema

A custom standalone subagent is defined by a single TOML file in `~/.codex/agents/` or `.codex/agents/` with at least three required fields:

- `name` (string) — stable identifier used for spawning and referencing.
- `description` (string) — human-facing text explaining when the agent should be used.
- `developer_instructions` (string) — core behavioral instructions for the agent (extended system prompt).

Optional fields include:

- `nickname_candidates` (string[]) — UI display aliases, no effect on logical identity.

- `model`, `model_reasoning_effort`, `sandbox_mode`, `mcp_servers`, `skills.config` — if omitted, inherit from the parent session.

Minimal TOML example:

```
name = "docs_researcher"
description = "Documentation specialist that verifies APIs and framework behavior."
developer_instructions = """
Use docs and exact references whenever possible.
Do not make code changes.
"""
```

Official documentation recommends narrow and opinionated agents: read-only/research subagents in parallel plus a single "writer" agent for implementation, avoiding multiple writers altering the same code in parallel.

3.3 Global Subagent Controls and Role Registration in `config.toml`

The `[agents]` section in `config.toml` controls global subagent parameters:

- `agents.max_threads` (default 6) — maximum number of concurrent agent threads.
- `agents.max_depth` (default 1) — maximum spawn nesting depth; root session starts at depth 0.
- `agents.job_max_runtime_seconds` — default timeout per worker in CSV fan-out.

The official recommendation is to keep `max_depth = 1` to avoid fan-out explosion, excessive token consumption, and unpredictable local resource usage.

Named roles can be registered via:

```
[agents.reviewer]
description = "Find correctness, security, and test risks in code."
config_file = "./agents/reviewer.toml"
nickname_candidates = ["Athena", "Ada"]
```

3.4 Sandbox, Approval, Skills, and MCP in Codex Subagents

Sandbox / Approval:

- `sandbox_mode`: `read-only`, `workspace-write`, or `danger-full-access`.
- `approval_policy`: `untrusted`, `on-request`, `never`, or a granular object controlling approvals for sandbox, rules, MCP, permissions, and skills.

Skills:

- `skills.config`: list of objects with `path` (pointing to `SKILL.md`) and `enabled` (bool) to enable/disable specific skills per agent.

MCP:

- STDIO support (`command`, `args`, `env`, `cwd`) and HTTP (`url`, headers, bearer, timeouts). Each server can have `enabled`, `required`, `enabled_tools`/`disabled_tools`, `scopes`, OAuth config, and timeouts.

This enables, for example, a `docs_researcher` subagent with a documentation MCP and `read-only` sandbox, while a `writer` inherits the MCP and enables `workspace-write` to create files.

3.5 Multi-Subagent Pipeline Examples in Codex

A well-designed four-agent pipeline for document production follows this pattern: `researcher`, `writer`, `reviewer`, and `curator`, each with a dedicated TOML and clear responsibilities. The OpenAI Agents SDK shows a similar pattern: a `Project Manager` orchestrates Designer, Frontend Developer, Backend Developer, and Tester agents using detailed instructions and explicit handoffs.

4. Skills in Codex: Structure and When to Use Them

4.1 Skill Directory Structure

```
my-skill/
  SKILL.md              # required
  scripts/              # optional (executable helpers)
  references/           # optional (docs, examples)
  assets/               # optional (templates, resources)
  agents/
    openai.yaml         # optional (UI metadata, policy, MCP dependencies)
```

`SKILL.md` contains mandatory YAML frontmatter with `name` and `description`, followed by detailed instructions defining the workflow. Skills can be invoked explicitly (`/skills`, `$skill-name`) or implicitly when the description matches the task. For this reason, descriptions should make clear both *when* and *when not* to trigger.

4.2 Metadata in `agents/openai.yaml`

The optional `agents/openai.yaml` adds interface metadata, policy, and dependencies:

- `interface.display_name`, `short_description`, `icon_small`, `icon_large`, `brand_color`, `default_prompt` — appearance and text for UI surfaces.
- `policy.allow_implicit_invocation` — controls whether the skill can be triggered automatically.
- `dependencies.tools` with `mcp` type entries — declares MCPs required for the workflow.

```
interface:
  display_name: "Release Check"
  short_description: "Verify release readiness before publishing."
  brand_color: "#3B82F6"
policy:
  allow_implicit_invocation: false
```

```
dependencies:
  tools:
    - type: "mcp"
      value: "openaiDeveloperDocs"
      transport: "streamable_http"
      url: "https://developers.openai.com/mcp"
```

This is the recommended way to package workflows that depend on external systems rather than overloading an agent's `developer_instructions`.

5. MCP in Codex: Role and Configuration

5.1 MCP's Role in the Codex Ecosystem

MCP is the protocol through which Codex communicates with external tools: browsers, databases, design systems, internal docs, etc. Codex treats MCP as first-class infrastructure: servers configured in `config.toml` are automatically exposed to the CLI, extension, and app, and can be attached to skills and agents.

The official Customization page emphasizes that the **skills + MCP** combination is where "it all comes together": skills define workflows, MCP connects those workflows to external systems, and subagents use both to operate in parallel.

5.2 MCP Configuration in `config.toml`

HTTP server example:

```
[mcp_servers.openaiDeveloperDocs]
url = "https://developers.openai.com/mcp"
startup_timeout_sec = 20
```

For STDIO, use `command`, `args`, `env`, `cwd` instead of `url`. Fields like `enabled`, `required`, `enabled_tools`, `disabled_tools`, `scopes`, `tool_timeout_sec`, and OAuth parameters allow controlling exposure and credentials.

6. Claude Code: Agents, Subagents, Skills, MCP, and Plugins

6.1 Extensibility Layers in Claude Code

Tutorials and consolidated Claude Code docs describe a stack very similar to Codex, but with its own nomenclature:

- CLAUDE.md: Analogous to AGENTS.md — global and per-project files defining persistent rules, conventions, commands, limits, and expectations.
- **Skills**: Instruction packages (often distributed via marketplaces like skills.sh or FastMCP) that Claude reads and applies automatically; used to define standards and specific

"superpowers."

- **MCP servers**: Connections to external services (GitHub, Brave, Playwright, Context7, File System, etc.) used for web access, browser automation, remote code access, and more.
- **Subagents / Agent Teams**: Specialized workers running in separate contexts, with their own prompts, permissions, and tools, capable of working independently or in parallel teams.
- **Plugins**: Packages combining skills, MCPs, subagents, and sometimes slash commands into distributable, reusable units.

6.2 Subagents in Claude Code

Official Claude Code documentation explains that subagents are specialized assistants that handle specific types of tasks, each with its own context, prompt, tool set, and permissions. When a task matches a subagent's description, Claude delegates to it; the subagent works independently and returns only results, saving context and keeping the main thread clean.

Key points about subagents in Claude Code:

- Each subagent runs in its own context window with a customized system prompt.
- Can have its own model configuration (Opus, Sonnet, Haiku), tools (MCPs), and permissions.
- Subagents **do not automatically inherit** skills from the parent; the full skill content is injected into the subagent when it is used.
- Explicit patterns:
  - **Parallel research** — multiple subagents exploring different areas, then Claude synthesizes.
  - **Chained subagents** — multi-step workflows where one subagent's output becomes the next one's input.

For larger or long-running workloads, Claude Code offers **agent teams**: multiple workers with persistent contexts, coordinated by a lead agent.

6.3 Skills in Claude Code

Guides highlight that skills in Claude Code are "instructions Claude can read on its own," used to define PR review standards, coding conventions, UX guidelines, etc. They work conceptually similar to Codex's Agent Skills: they define when they should be triggered and what to do. Skills typically include metadata and MCP dependencies (similar to `agents/openai.yaml` in Codex), and are often installed via marketplaces.

6.4 MCP in Claude Code

Claude Code uses MCP as the primary extension layer for external tools, exactly like Codex. Anthropic's article on multi-agent research systems shows a lead agent planning and creating research subagents that use MCPs in parallel (web, Google Workspace, integrations), then aggregating results. MCPs are configured via Claude Code's UI/CLI with environment variables for credentials and permission options — similar to Codex configuration, but with its own tooling.

6.5 Plugins in Claude Code

Guides describe plugins as an "extension layer" that combines skills (standards/routines), MCPs, subagents, and sometimes custom `/commands` into a single installable package. An example plugin might install code review standards + GitHub MCP + security and test-coverage subagents in one step.

Cross-platform bridge plugins also exist — such as `codex-cli-bridge`, which generates AGENTS.md from CLAUDE.md, provides commands to run Codex CLI from within Claude Code, and keeps both in sync.

6.6 Creation Structure / CLI (Overview)

The CLI guide emphasizes five main axes: **configuration**, **permissions**, **hooks**, **MCP**, and **subagents**.

Typical project flow:

1. Create CLAUDE.md (global + project) with rules and commands.
2. Install skills (via marketplace or manually).
3. Configure MCP servers (GitHub, Brave, etc.).
4. Define subagents/teams with descriptions, prompts, and tools.
5. Optionally package everything into a plugin.

7. Similarities Between Codex and Claude Code

Dimension	Codex	Claude Code
Persistent instruction file	AGENTS.md	CLAUDE.md
Reusable workflows	Skills (SKILL.md + openai.yaml)	Skills (marketplace, YAML metadata)
External tools/services	MCP (`config.toml` blocks)	MCP (UI/CLI config)
Parallel delegation	Subagents (TOML files)	Subagents / Agent Teams
Distribution unit	Skills + MCP configs (versioned repos)	Plugins (bundled install)

In both systems, the recommended engineering pattern is: **many read-only workers + one writer; narrow, opinionated agents; long procedures in skills, not in prompts; MCP for tool**

access; subagents for bounded parallelism.

8. Important Differences

8.1 Schema Formalization Level

- **Codex**: Detailed TOML schema for subagents and global config. Skills structure (`SKILL.md` + `agents/openai.yaml`) and MCP extremely well-specified in official docs.
- **Claude Code**: Concepts well-defined (subagents, skills, MCP, plugins, CLAUDE.md), but less public "schema reference" documentation; much of the practical knowledge comes from community guides and videos.

8.2 Distribution Unit

- **Codex**: Distribution revolves around skills and MCP servers; there is no official "plugin" concept bundling everything, though it is easy to version those folders in a repository.
- **Claude Code**: "Plugin" is a first-class citizen — a package bundling skills, MCPs, subagents, and configurations, installable in a few steps.

8.3 Interoperability

The `codex-cli-bridge` skill/plugin connects both worlds: it generates AGENTS.md from CLAUDE.md, exposes Codex CLI helpers inside Claude Code, and keeps both in sync. This confirms the mental model is close enough to allow near-direct portability between AGENTS.md ↔ CLAUDE.md and skills on both sides.

9. Creation Structures Side by Side

9.1 Creating a Subagent in Codex (Step-by-Step)

1. **Define baseline in** `~/.codex/config.toml`

```
model = "gpt-5.4"
approval_policy = "on-request"
sandbox_mode = "workspace-write"
web_search = "cached"

[agents]
max_threads = 6
max_depth = 1
```

2. **Create the agents directory**

```
mkdir -p .codex/agents
```

3. **Create one TOML file per agent** (e.g., `.codex/agents/reviewer.toml`)

```
name = "reviewer"
description = "PR reviewer focused on correctness, security, and missing tests."
model = "gpt-5.4"
model_reasoning_effort = "high"
sandbox_mode = "read-only"
developer_instructions = """
Review code like an owner.
Prioritize correctness, security, regressions, and missing test coverage.
"""
```

4. Register roles in `config.toml` (optional)

```
[agents.reviewer]
config_file = "./agents/reviewer.toml"
nickname_candidates = ["Athena", "Ada"]
```

5. Connect MCP if needed

```
[mcp_servers.openaiDeveloperDocs]
url = "https://developers.openai.com/mcp"
startup_timeout_sec = 20
```

6. Invoke via prompt

> "Spawn one subagent for security review, one for tests, and one for maintainability. Wait for all three and return a consolidated report."

7. Inspect threads via CLI

```
/agent
/mcp
```

9.2 Creating a Skill in Codex

1. Create the skill directory

```
mkdir -p .agents/skills/release-check
```

2. Create `SKILL.md`

```
---
name: release-check
description: Check release readiness before publishing — tests, docs, and changelog
---

## Steps
1. Run the full test suite.
2. Verify CHANGELOG.md has an entry for this version.
3. Confirm all public API changes are documented.
4. Check for any TODO/FIXME comments in changed files.
```

3. Add `agents/openai.yaml` (optional)

```
policy:
  allow_implicit_invocation: false
dependencies:
  tools:
    - type: "mcp"
      value: "github"
```

4. **Activate in** `config.toml`

```
[[skills.config]]
path = ".agents/skills/release-check/SKILL.md"
enabled = true
```

9.3 Creating Subagent/Skill/Plugin in Claude Code

1. **Create/edit** CLAUDE.md (global `~/.claude/CLAUDE.md` and per-project `CLAUDE.md`)

```
## Working agreements
- Always run tests before committing.
- Prefer TypeScript for new files.
- Ask before adding new production dependencies.
```

2. **Install skills** via marketplace or manually create skill folder with metadata and instructions.
3. **Configure MCP servers** via CLI (example: GitHub, Brave, Playwright) with env vars for credentials.
4. **Define subagents** with clear descriptions, system prompts, and specific tools/MCPs:
   - Use parallel research pattern for exploration tasks.
   - Use chained pattern for multi-step workflows.
   - Use agent teams for large, long-running workloads.
5. **Package as a plugin (optional)** combining skills, MCPs, and subagent definitions for one-click installation and sharing.

10. Experimental: CSV Batch Fan-Out in Codex

Codex supports an experimental workflow called `spawn_agents_on_csv` designed for repeated row-shaped work (not general recursive orchestration). Codex reads a CSV, spawns one worker per row, waits for completion, and exports combined results.

Useful for:

- One file/package/service per row.
- Repeated incident or PR audits.
- Structured summaries for many similar inputs.

Each worker must call `report_agent_job_result` exactly once; otherwise the row is marked with an error in the exported CSV.

11. Design Patterns and Anti-Patterns

Recommended Patterns

Pattern	Why It Works	Typical Config
Many readers, one writer	Keeps investigation parallel while avoiding conflicting edits	`explorer`/`reviewer` in `read-only`; one `worker` writes
Narrow, opinionated agents	Prevents role drift and keeps prompts predictable	Clear `description`, constrained `developer_instructions`
Docs specialist via MCP	Moves API verification out of the main agent	`read-only` + docs MCP + no-code-change instruction
Browser debugger before UI fix	Separates reproduction evidence from implementation	`browser_debugger` with high reasoning, then `ui_fixer`

Anti-Patterns to Avoid

- Do not create a single mega-agent that explores, reproduces, edits, reviews, and deploys.
- Do not let multiple writer agents edit overlapping code paths in parallel.
- Do not hide long procedures inside `developer_instructions` when they should be packaged as skills.
- Do not rely on project-scoped config in untrusted repos.

12. Practical Conclusions for Agent Engineering

The choice between Codex and Claude Code depends primarily on the use case:

- **Fine-grained declarative control with versioned schema**: Codex provides comprehensive, formally documented config (TOML subagents, `[agents]` controls, skills structure, MCP reference) that is ideal for platform engineering and CI/CD workflows.
- **Rich ecosystem, marketplace, and turnkey multi-agent teams**: Claude Code offers a more opinionated UX with skills, MCPs, agent teams, and plugins, making it faster to bootstrap complex setups from existing community packages.
- **Using both together**: The `codex-cli-bridge` plugin enables generating AGENTS.md from CLAUDE.md and running Codex CLI from within Claude Code, making it feasible to use Codex as the declarative runtime and Claude Code as the orchestrating IDE.

In either system, the modern standard for agent engineering is: **keep AGENTS/CLAUDE.md small and focused, put long procedures in skills, use MCP for tools, use subagents for well-bounded parallelism, and always prefer many readers over many writers.**